

1. Algoritmo A* (A estrella)

¿Qué es?

El algoritmo A* es un algoritmo de búsqueda utilizado en inteligencia artificial y ciencias de la computación para encontrar el camino más corto entre dos puntos dentro de un grafo o mapa.

Este algoritmo combina el costo real del recorrido con una estimación del costo restante para llegar al destino, lo que permite encontrar rutas eficientes de manera más rápida que otros algoritmos de búsqueda.

¿Cómo funciona?

El algoritmo A* funciona evaluando cada posible camino mediante una función que suma dos valores: el costo real desde el punto inicial hasta el nodo actual y una estimación del costo que falta para llegar al objetivo.

$$f(n)=g(n)+h(n)$$

Donde:

- $g(n)$ representa el costo real desde el nodo inicial hasta el nodo actual.
- $h(n)$ es la heurística, que estima el costo desde el nodo actual hasta el objetivo.
- $f(n)$ es el valor total que el algoritmo usa para decidir qué camino explorar.

El algoritmo analiza los nodos disponibles y siempre selecciona el que tenga el menor valor de $f(n)$ hasta encontrar la ruta óptima.

¿Para qué sirve?

El algoritmo A* se utiliza para resolver problemas de búsqueda de rutas óptimas. Algunas de sus aplicaciones más comunes son:

- Sistemas de navegación GPS para calcular rutas.
- Videojuegos, donde los personajes encuentran caminos en mapas.
- Robótica, para que los robots se desplacen evitando obstáculos.
- Inteligencia artificial, en problemas de planificación y búsqueda.

Ejemplo

Entrega de becas a la Universidad del Pacifico

En la ciudad de Buenaventura, un sistema de navegación para drones de entrega necesita encontrar la ruta más corta desde la Alcaldía hasta la Universidad del Pacífico. Para resolver este problema se utiliza el algoritmo de búsqueda A*, el cual combina el costo real del recorrido con una heurística que estima la distancia restante hasta el destino.

El sistema cuenta con un mapa simplificado de la ciudad representado como un grafo dirigido, donde cada nodo representa un lugar importante y cada arista representa una vía con un costo asociado que simboliza la distancia o tiempo de desplazamiento.

Nodo	Lugar
A	Alcaldía
B	Parque
C	Malecón
D	Bellavista
E	Kennedy
F	Universidad del Pacífico

Costos de desplazamiento entre lugares

Las conexiones entre los lugares y sus costos son:

- Alcaldía → Parque : 4
- Alcaldía → Malecón : 6
- Parque → Bellavista : 5
- Parque → Kennedy : 9
- Malecón → Bellavista : 3
- Bellavista → Kennedy : 2
- Bellavista → Universidad del Pacífico : 7
- Kennedy → Universidad del Pacífico : 3

Valores estimados al destino (Universidad del Pacífico)

Estos valores representan una estimación de la distancia restante desde cada nodo hasta el destino.

Nodo	Heurística
A	10
B	8
C	7
D	4
E	2
F	0

Aplicar el algoritmo A* para determinar la ruta óptima desde la Alcaldía (A) hasta la Universidad del Pacífico (F), minimizando el costo total del recorrido.

El programa debe mostrar la ruta encontrada, el costo total del camino, y además presentar una visualización animada del proceso, donde se van activando progresivamente los nodos y las aristas que conforman la ruta más corta.

README

Importación de librerías

```
import heapq

import networkx as nx

import matplotlib.pyplot as plt

from matplotlib.animation import FuncAnimation

from IPython.display import HTML
```

En esta parte se importan las librerías necesarias para el funcionamiento del programa.

- **heapq** permite implementar una **cola de prioridad**, que es fundamental para el algoritmo A*, ya que siempre selecciona el nodo con menor costo estimado.
- **networkx** se utiliza para crear y manipular grafos.
- **matplotlib** permite dibujar el grafo.
- **FuncAnimation** sirve para crear la animación del recorrido.
- **HTML** permite mostrar la animación como video dentro de un entorno interactivo como Jupyter Notebook.

Definición del grafo

```
grafo = {
```

```
'A': {'B': 4, 'C': 6},  
'B': {'D': 5, 'E': 9},  
'C': {'D': 3},  
'D': {'E': 2, 'F': 7},  
'E': {'F': 3},  
'F': {} }
```

Aquí se define el **grafo dirigido** mediante un diccionario.

Cada nodo representa un lugar y cada conexión indica el **costo de desplazamiento** entre dos puntos.

Por ejemplo:

- De **A** a **B** cuesta **4**
- De **A** a **C** cuesta **6**

Esto simula las distancias o el tiempo necesario para desplazarse entre lugares.

Definición de la heurística

```
heuristica = {  
    'A': 10,  
    'B': 8,  
    'C': 7,  
    'D': 4,  
    'E': 2,  
    'F': 0 }
```

La heurística representa una **estimación de la distancia restante hasta el destino**.

En este caso:

- Mientras más cerca esté un nodo de la **Universidad del Pacífico (F)**, menor será su valor heurístico.
- El nodo objetivo **F** tiene valor **0** porque ya se llegó al destino.

Esto permite que el algoritmo A* sea más eficiente al buscar el camino óptimo.

Nombres de los lugares

```
lugares = {  
    'A': 'A\nAlcaldía',  
    'B': 'B\nParque',  
    'C': 'C\nMalecón',  
    'D': 'D\nBellavista',  
    'E': 'E\nKennedy',  
    'F': 'F\nUniversidad\ndel Pacífico' }
```

Este diccionario se utiliza únicamente para **mostrar etiquetas más descriptivas en la visualización del grafo**.

Por ejemplo:

- $A \rightarrow$ Alcaldía
- $B \rightarrow$ Parque
- $C \rightarrow$ Malecón
- $D \rightarrow$ Bellavista
- $E \rightarrow$ Kennedy
- $F \rightarrow$ Universidad del Pacífico

El `\n` permite mostrar el texto en varias líneas dentro del nodo.

Creación del grafo visual

```
G = nx.DiGraph()
```

Se crea un **grafo dirigido**, ya que las conexiones tienen dirección.

Luego se agregan las aristas:

```
for nodo in grafo: for vecino, costo in grafo[nodo].items():  
    G.add_edge(nodo, vecino, weight=costo)
```

Cada conexión incluye su **peso**, que representa el costo de desplazamiento.

Posición de los nodos

```
pos = nx.spring_layout(G, k=5, seed=20)
```

Esto calcula automáticamente la posición de los nodos en el gráfico usando un algoritmo de distribución tipo resorte.

Esto se hace para que el grafo se vea **ordenado y fácil de entender visualmente**.

Implementación del algoritmo A*

La función principal es: `def a_star(inicio, objetivo):`

Este algoritmo busca la **ruta más corta desde un nodo inicial hasta un nodo objetivo**.

Cola de prioridad

```
cola = []  
  
heapq.heappush(cola, (0, inicio))
```

Aquí se crea una cola de prioridad donde siempre se selecciona el nodo con **menor costo estimado**.

Diccionarios de control

```
costos = {inicio: 0}  
  
padres = {inicio: None}
```

- **costos** guarda el costo acumulado para llegar a cada nodo.
- **padres** permite reconstruir la ruta final.

Búsqueda del camino

El algoritmo explora los nodos vecinos: `for vecino, costo in grafo[actual].items():`

Se calcula el nuevo costo: `nuevo_costo = costos[actual] + costo`

Luego se calcula la **prioridad**: `prioridad = nuevo_costo + heuristica[vecino]`

Esta es la fórmula principal del algoritmo A*: $f(n)=g(n)+h(n)$ $f(n) = g(n) + h(n)$ $f(n)=g(n)+h(n)$

Donde:

- **g(n)** = costo real recorrido
- **h(n)** = estimación heurística
- **f(n)** = costo total estimado

Reconstrucción de la ruta

Después de encontrar el objetivo, se reconstruye la ruta:

```
while nodo:

    ruta.append(nodo)

    nodo = padres[nodo]
```

Luego se invierte: `ruta.reverse()`

Esto permite obtener la ruta desde el inicio hasta el destino.

Ejecución del algoritmo

```
ruta_final, costo_total = a_star('A', 'F')
```

Aquí se busca la mejor ruta desde **A (Alcaldía)** hasta **F (Universidad del Pacífico)**.

Se imprime el resultado:

```
print("Ruta óptima encontrada:")

print(" → ".join(ruta_final))

print("Costo total:", costo_total)
```

Visualización del grafo

Se crea la figura: `fig, ax = plt.subplots()`

Luego se define la función que actualiza cada frame de la animación.

Animación del recorrido

La función: `def actualizar(frame):`

Controla cómo se va mostrando el camino.

Primero se iluminan **los nodos**: `nodos_activos = (frame // 2) + 1`

Luego las **aristas del camino**: `aristas_activas = frame // 2`

Esto genera el efecto visual de: Nodo → Arista → Nodo → Arista, hasta completar toda la ruta.

Creación de la animación

```
anim = FuncAnimation(
```

```
fig,  
  
actualizar,  
  
frames=len(ruta_final)*2,  
  
interval=1200 )
```

- Cada paso dura **1200 milisegundos**.
- Se generan el doble de frames para alternar entre nodos y aristas.

Finalmente se muestra con video: `HTML(anim.to_html5_video())`

Resultado esperado

El programa mostrará:

- La **ruta óptima encontrada**
- El **costo total**
- Una **animación del recorrido del camino más corto**

Ejemplo de salida:

Ruta óptima encontrada:

A → C → D → E → F

Costo total: 14

2. Heurística

¿Qué es?

Es una regla práctica que permite tomar decisiones rápidas usando estimaciones o experiencias previas, sin analizar todas las opciones posibles, en muchos problemas de IA, la heurística ayuda a encontrar una solución más rápido, guiando al algoritmo hacia las opciones que parecen más prometedoras.

La heurística es un método o función que permite estimar la distancia o el costo restante para llegar a una meta, sin necesidad de calcular el camino exacto.

En algoritmos de búsqueda como A*, la heurística ayuda a orientar la búsqueda hacia el objetivo de manera más eficiente.

¿Cómo funciona?

La heurística funciona proporcionando una estimación del costo que falta para llegar al objetivo desde un nodo determinado.

Esta estimación permite que el algoritmo priorice los caminos que parecen más prometedores, reduciendo la cantidad de nodos que deben explorarse.

Por ejemplo, en un mapa, una heurística podría calcular la distancia aproximada entre dos puntos, ayudando al algoritmo a decidir qué dirección seguir.

¿Para qué sirve?

La heurística sirve para:

- Reducir el tiempo de búsqueda en algoritmos.
- Optimizar el proceso de encontrar soluciones.
- Guiar algoritmos de inteligencia artificial hacia las mejores opciones posibles.

Se utiliza en:

- algoritmos de búsqueda
- inteligencia artificial
- optimización
- videojuegos
- sistemas de navegación

Ejemplo #1

La mejor ruta para llegar al Puerto de Buenaventura

En la ciudad de Buenaventura, un conductor necesita llegar lo más rápido posible al Puerto de Buenaventura para entregar mercancía.

Existen tres rutas posibles para llegar al puerto, pero el conductor no conoce el tiempo exacto que tardará en cada una debido al tráfico y a las condiciones de la vía.

Para tomar una decisión rápida, el sistema utiliza una heurística, que consiste en estimar el costo de cada ruta teniendo en cuenta dos factores:

- Distancia de la ruta (km)
- Nivel de tráfico estimado (valor entre 1 y 20)

La heurística utilizada es:

Costo estimado = distancia + tráfico

El sistema debe calcular el valor heurístico de cada ruta y seleccionar la ruta con el menor costo estimado, ya que probablemente será la más rápida.

El objetivo es determinar cuál es la mejor ruta usando una función heurística en Python.

DATOS DEL PROBLEMA

Ruta	Distancia (km)	Tráfico estimado
Ruta Centro	10	15
Ruta Alterna	14	5
Ruta Malecón	12	10

README

Definición de las rutas

```
rutas = { "Ruta Centro": {"distancia": 10, "trafico": 15},  
rutas = { "Ruta Centro": {"distancia": 10, "trafico": 15},  
"Ruta Alterna": {"distancia": 14, "trafico": 5},  
"Ruta Malecon": {"distancia": 2, "trafico": 10} }
```

Aquí se crea un diccionario llamado `rutas` donde se guardan las posibles rutas para llegar al puerto, cada ruta tiene dos valores:

- `distancia` → qué tan larga es la ruta
- `trafico` → nivel de congestión vehicular

Permitiendo que el programa **compare las rutas para elegir la mejor**.

Función heurística: `def heuristica(ruta):`

Aquí se define una **función heurística**.

En Inteligencia Artificial, una **heurística es una estimación rápida del costo de una solución** para tomar decisiones.

Obtención de datos de la ruta

```
distancia = rutas[ruta]["distancia"]  
trafico = rutas[ruta]["trafico"]
```

La función busca en el diccionario:

- la **distancia**
- el **tráfico**

de la ruta que se esté evaluando.

Por ejemplo, si la ruta es **Ruta Centro**, el programa obtiene:

```
distancia = 10
trafico = 15
```

Cálculo del costo estimado: `costo_estimado = distancia + trafico`

Se calcula un **costo heurístico** sumando: `Costo = Distancia + Trafico`

Esto representa una **estimación del tiempo o dificultad de la ruta**.

Retorno del costo

```
return costo_estimado
```

La función devuelve el costo calculado para que el programa pueda **comparar las rutas**.

Selección de la mejor ruta: `mejor_ruta = min(rutas, key=heuristica)`

Aquí Python busca la **ruta con el menor costo** `min()` encuentra el valor más pequeño y usa la función `heuristica` para evaluarlo.

El proceso sería:

```
Ruta Centro    -> costo 25
Ruta Alterna   -> costo 19
Ruta Malecón   -> costo 12
```

Por lo tanto: `Mejor ruta = Ruta Malecón`

Mostrar resultados

```
print("La mejor ruta para llegar al Puerto de Buenaventura es:",
mejor_ruta)
print("Fue la mejor ruta con un costo de:", heuristica(mejor_ruta))
```

El programa muestra: `La mejor ruta para llegar al Puerto de Buenaventura es: Ruta Malecon`

`Fue la mejor ruta con un costo de: 12`

Esto significa que **la Ruta Malecón tiene el menor costo estimado** considerando distancia y tráfico.

Ejemplo #2

La mejor ruta para llegar al Parque del Cafe

Un conductor sale desde la ciudad de Buenaventura y quiere viajar hasta el Parque del Café, uno de los destinos turísticos más visitados del eje cafetero.

Para llegar al destino existen tres posibles rutas, pero el conductor no conoce exactamente cuánto tiempo tardará en cada una, ya que durante el trayecto pueden existir factores como tráfico, estado de la carretera o número de peajes.

Para tomar una decisión rápida, el sistema utiliza una heurística, que consiste en estimar el costo de cada ruta utilizando varios factores del viaje.

La heurística utilizada será:

Costo estimado = distancia + (tráfico × 2) + peajes

Se multiplica el tráfico por 2 porque el tráfico suele afectar más el tiempo del viaje que otros factores.

El sistema debe calcular el valor heurístico de cada ruta y escoger la ruta con menor costo estimado, ya que probablemente será la más rápida y conveniente.

El objetivo es determinar cuál es la mejor ruta usando una función heurística programada en Python.

DATOS DEL PROBLEMA

Ruta	Distancia (km)	Nivel de tráfico (1-10)	Número de peajes
Ruta Buenaventura – Cali – Armenia	210	6	3
Ruta Buenaventura – Buga – Tuluá – Armenia	220	4	2
Ruta Buenaventura – Palmira – Armenia	230	3	2

README

Definición de las rutas

```
 rutas = { "Buenaventura - Cali - Armenia": { "distancia":  
210,"trafico": 6,"peajes": 3 }, "Buenaventura - Buga - Tuluá -  
Armenia": { "distancia": 220, "trafico": 4,"peajes": 2
```

```
}, "Buenaventura - Palmira - Armenia": {"distancia": 230, "trafico": 3, "peajes": 2 } }
```

Aquí se crea un **diccionario** llamado **rutas** que contiene tres posibles caminos para ir desde **Buenaventura al Parque del Café**.

Cada ruta tiene tres variables:

- **distancia** → kilómetros aproximados del recorrido
- **trafico** → nivel de congestión de la vía
- **peajes** → cantidad de peajes en el trayecto

Esto permite que el programa **compare diferentes factores de la ruta**.

Función heurística

```
def heuristica(ruta):
```

Esta función se encarga de **calcular el costo estimado de cada ruta**.

En Inteligencia Artificial una **heurística es una estimación rápida para elegir la mejor opción sin calcular todo exactamente**.

Obtención de los datos de la ruta

```
distancia = rutas[ruta]["distancia"]
```

```
trafico = rutas[ruta]["trafico"]
```

```
peajes = rutas[ruta]["peajes"]
```

Estas líneas toman los valores del diccionario.

Ejemplo si la ruta es: **Buenaventura - Cali - Armenia**

Entonces el programa obtiene:

```
distancia = 210
```

```
trafico = 6
```

```
peajes = 3
```

Cálculo del costo heurístico: `costo = distancia + (trafico * 2) + peajes`

Aquí se calcula el **costo total estimado de la ruta**.

La fórmula usada es:

$\text{Costo} = \text{Distancia} + (\text{Tráfico} \times 2) + \text{Peajes}$

Se multiplica el tráfico por 2 porque **el tráfico afecta más el tiempo de viaje**.

Retorno del costo: `return costo`

La función devuelve el costo para que el programa **pueda comparar las rutas**.

Línea que realiza el ciclo de comparación: `mejor_ruta = min(rutas, key=heuristica)`

Esta es la línea más importante; aquí Python hace **un ciclo automático por todas las Rutas**, internamente Python hace algo equivalente a esto:

```
for ruta in rutas: heuristica(ruta)
```

Es decir:

1. Evalúa la primera ruta
2. Calcula su costo
3. Evalúa la segunda
4. Calcula su costo
5. Evalúa la tercera
6. Compara todos los resultados

Luego **elige la ruta con el menor costo**.

Proceso que hace Python:

Ruta 1 → costo 225

Ruta 2 → costo 230

Ruta 3 → costo 238

Entonces: `Mejor ruta = Buenaventura - Cali - Armenia`

Mostrar el resultado

```
print("La mejor ruta para llegar al Parque del Café es:",  
mejor_ruta)
```

```
print("Fue la mejor ruta con un costo de:", heuristica(mejor_ruta))
```

Estas líneas se muestran en pantalla:

La mejor ruta para llegar al Parque del Café es: Buenaventura - Cali
- Armenia

Fue la mejor ruta con un costo de: 225

3. Teoría de Juegos

¿Qué es?

La teoría de juegos es una rama de las matemáticas y la economía que estudia cómo las personas o agentes toman decisiones estratégicas cuando el resultado depende también de las decisiones de otros participantes.

Analiza situaciones donde varios jugadores interactúan y cada uno busca maximizar su beneficio o minimizar sus pérdidas.

¿Cómo funciona?

La teoría de juegos funciona mediante el análisis de modelos de interacción estratégica.

En cada juego se consideran tres elementos principales:

- Jugadores: quienes toman decisiones.
- Estrategias: las opciones disponibles para cada jugador.
- Resultados o recompensas: lo que cada jugador gana o pierde dependiendo de las decisiones tomadas.

El objetivo es analizar qué estrategia es más conveniente considerando las decisiones posibles de los demás jugadores.

¿Para qué sirve?

La teoría de juegos se utiliza para analizar y predecir comportamientos en situaciones de competencia o cooperación.

Se aplica en áreas como:

- Economía, para estudiar mercados y competencia entre empresas.
- Política, para analizar negociaciones y conflictos.

- Inteligencia artificial, para desarrollar agentes que tomen decisiones estratégicas.
- Biología, para estudiar comportamientos evolutivos.

Ejemplo #1

Competencia entre D1 y Ara

En una ciudad de Colombia, los supermercados Tiendas D1 y Tiendas Ara compiten por atraer más clientes en el mismo barrio. Ambos supermercados venden productos similares como arroz, leche y huevos.

Cada supermercado tiene dos estrategias posibles:

- Bajar precios para atraer más clientes.
- Mantener precios para conservar su margen de ganancia.

Sin embargo, la ganancia de cada supermercado depende también de lo que haga el competidor:

- Si ambos mantienen precios, ambos obtienen ganancias normales.
- Si uno baja precios y el otro no, el que baja precios atrae más clientes y gana más.
- Si ambos bajan precios, ambos venden más pero con menor margen de ganancia.

El objetivo es analizar qué estrategia conviene más utilizando teoría de juegos y encontrar el mejor resultado estratégico para ambos supermercados.

Modelo del juego (Matriz de pagos)

	Ara mantiene precio	Ara baja precio
D1 mantiene precio	(8, 8)	(3, 10)
D1 baja precio	(10, 3)	(5, 5)

Donde:

- El primer número representa la ganancia de D1.
- El segundo número representa la ganancia de Ara.
- (8,8) → ambos mantienen precios y ganan bien.

- $(10,3) \rightarrow$ D1 baja precios y gana más clientes.
- $(3,10) \rightarrow$ Ara baja precios y gana más clientes.
- $(5,5) \rightarrow$ ambos bajan precios y ganan menos.

Ejemplo #2

Dos conductores buscando parqueadero en el Hotel Torremar

En el parqueadero del Hotel Torremar solo queda un espacio cercano a la entrada y varios espacios más lejanos. Dos conductores (A y B) llegan al mismo tiempo y deben decidir rápidamente entre dos estrategias:

- Intentar tomar el parqueadero cercano (más cómodo, pero puede generar conflicto si ambos lo buscan).
- Ir directamente a un parqueadero lejano (menos cómodo, pero siempre disponible).

Cada conductor sabe que su resultado depende también de lo que decida el otro.

MATRIZ DE BENEFICIOS

	Conductor B cercano	Conductor B lejano
A cercano	$(2, 2)$	$(9, 4)$
A lejano	$(4, 9)$	$(6, 6)$

- $(2,2) \rightarrow$ ambos intentan el cercano y pierden tiempo discutiendo o esperando.
- $(9,4) \rightarrow$ A toma el cercano y B debe ir al lejano.
- $(4,9) \rightarrow$ B toma el cercano.
- $(6,6) \rightarrow$ ambos van al lejano sin conflicto.

README

Importación de la librería

```
import numpy as np
```

Aquí se importa la librería **NumPy**, que normalmente se usa para trabajar con matrices y cálculos numéricos en Python.

Sin embargo, en este código **no se utiliza realmente**, por lo que el programa funcionaría igual si se elimina esta línea.

Definición de las estrategias

```
estrategias = ["Mantener precio", "Bajar precio"]
```

Se crea una **lista llamada estrategias** que contiene las dos decisiones posibles para los supermercados:

1. **Mantener precio**
2. **Bajar precio**

Estas estrategias serán usadas por:

- **D1**
- **Ara**

Definición de la matriz de pagos

```
matriz = [  
    [(8,8), (3,10)],  
    [(10,3), (5,5)] ]
```

Aquí se define la **matriz de pagos del juego**.

Cada posición representa las **ganancias de ambos jugadores** dependiendo de las decisiones que tomen.

El formato es: (D1 , Ara)

La matriz queda así: (8,8)

significa: D1 gana 8 Y Ara gana 8

Mostrar la matriz de pagos

```
print("Matriz de pagos (D1, Ara):\n")
```

Este **print** solo muestra un título para organizar la salida del programa.

Primer ciclo for

```
for i in range(2):  
    for j in range(2):
```

Aquí aparecen **dos ciclos anidados: range(2)**

Significa que el ciclo recorrerá los valores: 0 , 1

Esto se hace porque hay **2 estrategias posibles**.

Funcionamiento del ciclo

El programa recorre **todas las combinaciones de estrategias**.

Mostrar cada combinación

```
print(f"D1: {estrategias[i]} | Ara: {estrategias[j]} -> {matriz[i][j]}")
```

Aquí se imprime cada escenario posible del juego.

D1: Mantener precio | Ara: Mantener precio -> (8,8)

D1: Mantener precio | Ara: Bajar precio -> (3,10)

D1: Bajar precio | Ara: Mantener precio -> (10,3)

D1: Bajar precio | Ara: Bajar precio -> (5,5)

Mostrar resultados del juego `print("\nResultados posibles del juego:\n")`

Este `print` separa la segunda parte del programa, donde se analizan los resultados.

Segundo ciclo `for`

```
for i in range(2):  
    for j in range(2):
```

Se repite el mismo ciclo para **analizar cada resultado individualmente**.

Separación de ganancias `d1, ara = matriz[i][j]`

Aquí se extraen los valores de la matriz, ejemplo: (8,8)

Se divide en: `d1 = 8 , ara = 8`

Esto permite imprimir las ganancias de cada jugador por separado.

Mostrar el resultado de cada estrategia

```
print(f"Estrategia elegida:")  
print(f"D1 -> {estrategias[i]}")  
print(f"Ara -> {estrategias[j]}")  
print(f"Ganancia D1 = {d1}")  
print(f"Ganancia Ara = {ara}")
```

Esto muestra claramente:

- qué estrategia eligió cada supermercado
- cuánto gana cada uno

Ejemplo:

```
Estrategia elegida:  
D1 -> Mantener precio  
Ara -> Bajar precio  
Ganancia D1 = 3  
Ganancia Ara = 10
```

Separador visual `print("-----")`

Solo se usa para **ordenar mejor la salida del programa**.

Ejemplo #3

Dos colectivas compitiendo por recoger pasajeros

En una avenida de Buenaventura, dos colectivas de transporte público circulan por la misma ruta y se aproximan a un paradero donde hay varios pasajeros esperando. Cada conductor quiere ser el primero en detenerse para recogerlos.

Cada colectiva tiene dos posibles estrategias:

- Frenar inmediatamente para intentar recoger a los pasajeros primero.
- Seguir avanzando un poco más para mantener la velocidad de la ruta y evitar retrasos.

Sin embargo, el resultado depende de lo que haga la otra colectiva:

- Si ambas frenan al mismo tiempo, se genera congestión y ambas pierden tiempo.
- Si una frena primero y la otra sigue, la que frena recoge más pasajeros.
- Si ambas siguen avanzando, ninguna pierde tiempo pero los pasajeros pueden esperar más.

Este escenario puede analizarse mediante teoría de juegos, ya que cada conductor toma su decisión considerando lo que podría hacer el otro.

MATRIZ

	Colectiva B frena	Colectiva B sigue
Colectiva A frena	(4, 4)	(12, 3)
Colectiva A sigue	(3, 11)	(7, 8)

- (4,4) → ambas frenan y se estorban entre sí.
 (12,3) → A frena primero y recoge la mayoría de pasajeros.
 (3,11) → B frena primero y gana más pasajeros.
 (7,8) → ambas siguen y los pasajeros se distribuyen.

README

Definición de las estrategias

`estrategias = ["Frenar para recoger pasajeros", "Seguir avanzando"]`

Aquí se crea una lista llamada **estrategias**.

Esta lista contiene las dos decisiones posibles que puede tomar cada colectiva:

1. **Frenar para recoger pasajeros**
2. **Seguir avanzando**

Estas decisiones las pueden tomar:

- **Colectiva A**
- **Colectiva B**

Definición de la matriz de pagos

```
matriz = [  
    [(4,4), (12,3)],    # A frena  
    [(3,11), (7,8)]     # A sigue    ]
```

Aquí se define la **matriz de pagos del juego**.

Cada número representa la **ganancia o beneficio** de cada colectiva dependiendo de las decisiones que tomen.

El formato de cada par es: (A , B)

Es decir:

- primer número → ganancia de **Colectiva A**
- segundo número → ganancia de **Colectiva B**

La matriz quedaría así: (12,3) significa que:

- Colectiva **A gana 12**
- Colectiva **B gana 3**

porque A recogió más pasajeros.

Título del programa `print("Competencia entre dos colectivas por pasajeros\n")`

Esta línea solo imprime un **título** para explicar el escenario del juego.

En este caso se simula **la competencia entre dos buses por recoger pasajeros en la misma ruta**.

Mostrar la matriz de pagos `print("Matriz de pagos (A,B):\n")`

Este `print` muestra el encabezado de la matriz de resultados.

Ciclos for para recorrer la matriz

```
for i in range(2):  
    for j in range(2):
```

Aquí aparecen **dos ciclos for anidados**: `range(2)`

Significa que el ciclo recorrerá: 0 , 1

Esto ocurre porque hay **dos estrategias posibles**.

Mostrar cada combinación

```
print(f"A: {estrategias[i]} | B: {estrategias[j]} -> {matriz[i][j]}")
```

Esta línea imprime cada escenario posible: ejemplo de salida:

```
A: Frenar para recoger pasajeros | B: Frenar para recoger pasajeros  
-> (4,4)
```

```
A: Frenar para recoger pasajeros | B: Seguir avanzando -> (12,3)
```

```
A: Seguir avanzando | B: Frenar para recoger pasajeros -> (3,11)
```

```
A: Seguir avanzando | B: Seguir avanzando -> (7,8)
```

Esto permite ver **qué ganancia obtiene cada colectiva en cada situación**.

Sección de análisis estratégico: `print("\nAnálisis estratégico:\n")`

Esta línea introduce la parte donde se **analiza cuál estrategia es mejor para cada co.**

Análisis de decisiones

```
print("Si B frena, A prefiere frenar (4 > 3)")
```

```
print("Si B sigue, A prefiere frenar (12 > 7)")
```

Aquí se analiza la decisión de **Colectiva A**.

Caso 1: Si B frena:

- A frena → gana **4**
- A sigue → gana **3**

Entonces A prefiere **frenar**.

Caso 2: Si B sigue:

- A frena → gana **12**
- A sigue → gana **7**

A nuevamente prefiere **frenar**; por lo tanto, **frenar es la mejor estrategia para A**.

Análisis de la colectiva B

```
print("Si A frena, B prefiere frenar (4 > 3)")  
print("Si A sigue, B prefiere frenar (11 > 8)")
```

Aquí se analiza la decisión de **Colectiva B**.

Caso 1: Si A frena:

- B frena → gana 4
- B sigue → gana 3

B prefiere **frenar**.

Caso 2: Si A sigue:

- B frena → gana 11
- B sigue → gana 8

B también prefiere **frenar**.

Entonces **frenar es también la mejor estrategia para B**.

Equilibrio del juego

```
print("\nEquilibrio encontrado:")  
print("Ambas colectivas deciden frenar -> resultado (4,4)")
```

Aquí se muestra la conclusión del juego; en **teoría de juegos**, este punto se llama:

Equilibrio de Nash.

Esto ocurre cuando **ningún jugador puede mejorar su resultado cambiando su estrategia de forma individual**.

En este caso:

A	B	Resultado
Frenar	Frenar	(4,4)

Ambas colectivas frenan porque es la decisión más segura para cada una.